# Brief Papers

## Policy-Adjustable Q-Learning for Data-Driven Nonlinear Optimal Tracking Control

Jiaoyuan Chen, *Graduate Student Member, IEEE*, Dawei Gong, Yuyang Zhao, Shijie Song, and Minglei Zhu, *Member, IEEE*

***Abstract*—This article investigates a novel policy-adjustable Q-learning (PA-QL) algorithm aimed at addressing the optimal tracking control (OTC) problem for nonlinear discrete-time (DT) systems with enhanced adaptability and flexibility. A novel iteration scheme is developed that integrates the control weights into the augmented neural network (NN) input, thereby reformulating the learning process to explicitly characterize the optimal policy as a function of the adjustable weights. Consequently, the learned control policy is not constrained by predetermined weights, allowing for dynamic adjustment after offline training is completed. Moreover, such adjustments can be performed online seamlessly, offering substantially greater flexibility in adapting to changes in operating conditions or control objectives. Finally, the effectiveness of the proposed algorithm is established through rigorous theoretical analysis and further validated by simulation studies.**

***Index Terms*—Adaptive dynamic programming (ADP), neural network (NN), optimal tracking control (OTC), policy adjustable, Q-learning.**

## I. Introduction

Reinforcement learning (*RL*) has attracted extensive attention in the control community because it can obtain optimal policies from input–output data, either directly or indirectly, without requiring explicit mechanistic models of the system [1], [2], [3], [4]. Adaptive dynamic programming (ADP) has been established as a prominent *RL*-based framework for addressing optimal control problems, particularly optimal tracking control (OTC) in nonlinear systems [5], [6], [7], [8], [9]. Within the framework of ADP, the action-dependent heuristic dynamic programming (ADHDP), also known as Q-learning, has attracted substantial interest because it directly approximates the action–value function, thereby facilitating the determination of the optimal control policy [10], [11], [12], [13].

The Q-learning algorithm was initially introduced for solving optimal control problems of linear systems [14], [15], [16], [17], [18] and was subsequently extended to nonlinear systems to broaden its applicability [19], [20], [21], [22]. A critic-only Q-learning scheme was proposed in [19], which reduces the computational burden and accelerates the learning process. In [20], Q-learning was primarily investigated in the context of historical datasets and offline training, with the main focus placed on convergence analysis. An off-policy variant of Q-learning, which enhances data utilization, was later proposed in [21]. Building on the ideas of [20] and [21], [22] developed DsoQL, which incorporates a model neural network (NN) to significantly enhance the data robustness and convergence stability of Q-learning. In addition, Q-learning as well as other ADP methods has demonstrated extensive applicability across diverse domains, ranging from process management in wastewater treatment [23], [24], energy management of power systems [25], [26], [27], and path planning and tracking control of autonomous systems [28], [29], [30].

However, in practical applications, adjusting the utility function weights to accommodate changes in environmental conditions or control objectives is crucial, which is a capability readily available in classical optimal control methods such as linear quadratic regulator (LQR) and model predictive control (MPC). Such adaptation can be achieved through error-based rules, optimization algorithms, or data-driven strategies [31], [32], [33], all of which benefit from the real-time optimization nature of these methods. In contrast, ADP-based approaches typically require offline training with fixed cost function, resulting in the learned policy that is closely tied to the original design. Once training is complete, modifying the control policy generally necessitates retraining, making flexible and task-adaptive cost regulation difficult to achieve in traditional ADP frameworks.

To endow the algorithm with such capability, a straightforward approach is to pretrain multiple control policies using the traditional Q-learning algorithm under different task requirements. During system operation, the controller can then switch among these pretrained policies according to the current control demand [34], [35]. However, this method suffers from several limitations. Switching among pretrained policies may introduce uncertain effects on system stability; the need for repeated training significantly increases computational cost; and the adjustment remains discrete intrinsically, limiting its applicability to a finite set of predefined scenarios. To overcome these challenges, this article proposes a novel policy-adjustable Q-learning (PA-QL) algorithm that transforms the learning task from solving a specific optimization problem to learning a generalized policy representation. The main contributions of this work are as follows.

1) Unlike traditional Q-learning limited to fixed cost functions [19], [20], [21], [22], this article proposes a PA-QL framework that augments the NN input with the adjustable control weights. By explicitly modeling the optimal policy as a function of adjustable control weights, this architecture effectively bridges the gap between ADP and classical online optimal control, enabling the learned policy to inherit the online weight adaptation capabilities typically reserved for classical online optimization-based methods.
2) In contrast to conventional methods that rely on switching among a discrete set of pretrained policies, the proposed algorithm achieves continuous parametric generalization within a single training. The NN learns to approximate a continuous

Received 22 April 2025; revised 8 October 2025 and 4 January 2026; accepted 4 March 2026. Date of publication 16 March 2026; date of current version 2 September 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62303095 and Grant U2568231, in part by the Fundamental Research Funds for the Central Universities under Grant 2682025CX080 and Grant 2682025XJ007, and in part by the Natural Science Foundation of Sichuan under Grant 2026YFHZ0235 and Grant 2026YFHZ0221. *(Corresponding author: Dawei Gong.)*

Jiaoyuan Chen and Yuyang Zhao are with the School of Mechanical and Electrical Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: chen.jiaoyuan@std.uestc.edu.cn; yuyangzhao1994@std.uestc.edu.cn).

Dawei Gong is with the School of Mechanical and Electrical Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China, and also with the School of Electronic Information and Electrical Engineering, Chengdu University, Chengdu 610106, China (e-mail: pzhzhx@126.com).

Shijie Song and Minglei Zhu are with the Institute of Smart City and Intelligent Transportation, Southwest Jiaotong University, Chengdu 611756, China (e-mail: shijie.song@swjtu.edu.cn; minglei.zhu@swjtu.edu.cn).

Digital Object Identifier 10.1109/TNNLS.2026.3672136

mapping of optimal policies over the weight space, enabling seamless online adjustment of control policy. This approach not only avoids the stability risks associated with abrupt policy switching but also drastically reduces the computational complexity involved in training multiple policy networks.

3) The developed PA-QL algorithm is designed for a wide range of input-affine nonlinear discrete-time (DT) systems. It employs a data-driven structure based on a model NN, relying entirely on I/O data and avoiding the need for explicit mechanistic models to enhance adaptability to complex real-world scenarios.

The rest of this article is organized as follows. Section II presents the problem formulation. Section III develops the PA-QL algorithm. Section IV provides the convergence analysis of PA-QL algorithm. Section V presents two simulation cases to validate both the effectiveness and the optimality of the proposed algorithm. Section VI concludes this article.

*Notations:* The operation region is denoted as $\Omega$ and $\mathbb{R}^n$ denotes the $n$ dimensional Euclidean space.

## II. Problem Formulation

This article explores the input-affine nonlinear DT systems as follows:

$$x_{k+1} = \mathbb{F}(x_k, u_k) = f(x_k) + g(x_k)u_k \tag{1}$$

where $k \in \mathbb{N}$ is the time index; $x_k \in \mathbb{R}^n$ is the system state vector; $u_k \in \mathbb{R}^m$ denotes the system control input vector; $\mathbb{F}(\cdot) : \mathbb{R}^n \times \mathbb{R}^m \mapsto \mathbb{R}^n$ is an unknown system function; and $g(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}^{n\times m}$ and $f(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}^n$ are the input dynamics and the drift dynamics of the system, respectively. The system is controllable on a set $\Omega_u \subset \mathbb{R}^m$. Assume that $\mathbb{F}(\cdot)$ is Lipschitz continuous on a compact set $\Omega_x \subset \mathbb{R}^n$ and $\mathbb{F}(0, 0) = 0$ is the unique equilibrium of the system.

For the infinite-time optimal tracking problem, the objective is to seek the optimal control policy $u^*(x_k) \in \Omega_u$ for system (1), which can ensure that the state $x_{k+1}$ pursues the desired trajectory $d_{k+1} \in \mathbb{R}^n$ generated by

$$d_{k+1} = \Xi(d_k) \tag{2}$$

where $\Xi(\cdot) : \mathbb{R}^n \mapsto \mathbb{R}^n$ is differentiable on a compact set $\Omega_d \subset \mathbb{R}^n$. Furthermore, we assume the presence of a steady control $u_s(d_k)$ that complies with the following equation:

$$d_{k+1} = \mathbb{F}(d_k, u_s(d_k)) = f(d_k) + g(d_k)u_s(d_k). \tag{3}$$

Then, the steady control is derived as

$$u_s(d_k) = g^{-1}(d_k)\big(\Xi(d_k) - f(d_k)\big). \tag{4}$$

The existence of steady control can be guaranteed under the following assumption.

*Assumption 1:* $g(\cdot)$ is a full-rank square matrix and $g^{-1}(\cdot)$ is its inverse matrix, which satisfies $g(\cdot)g^{-1}(\cdot) = g^{-1}(\cdot)g(\cdot) = I_n$, where $I_n$ is the $n$-dimensional identity matrix.

To achieve the infinite-time optimal tracking, the tracking error is derived as

$$e_k = x_k - d_k. \tag{5}$$

Then, the tracking control can be expressed as

$$v_k = u(x_k) - u_s(d_k). \tag{6}$$

By integrating (1)–(6), a new augmented system can be derived as

$$\xi_{k+1} = F(\xi_k) + G(\xi_k)v_k \tag{7}$$

where $\xi_k = [e_k^T, d_k^T]^T \in \mathbb{R}^{2n}$ denotes the state vector and $v_k \in \mathbb{R}^m$ represents control input vector of the augmented system. $F(\cdot) : \mathbb{R}^{2n} \mapsto \mathbb{R}^{2n}$ and $G(\cdot) : \mathbb{R}^{2n} \mapsto \mathbb{R}^{2n\times m}$ can be expressed as

$$F(\xi_k) = \begin{bmatrix} f(e_k + d_k) + g(e_k + d_k)u_s(d_k) - \Xi(d_k) \\ \Xi(d_k) \end{bmatrix} \tag{8}$$

$$G(\xi_k) = \begin{bmatrix} g(e_k + d_k) \\ 0_{n\times m} \end{bmatrix}. \tag{9}$$

The cost function for the augmented system (7) in the infinite-time OTC problem is defined by the following expression:

$$\begin{aligned} \mathcal{V}(\xi_k) &= \sum_{j=k}^{\infty} \left\{ [e_j^T, d_j^T] \begin{bmatrix} \mathcal{Q}_{in} & 0_{n\times n} \\ 0_{n\times n} & 0_{n\times n} \end{bmatrix} \begin{bmatrix} e_j \\ d_j \end{bmatrix} + v_j^T \mathcal{R} v_j \right\} \\ &= \sum_{j=k}^{\infty} \xi_j^T \mathcal{Q} \xi_j + v_j^T \mathcal{R} v_j = \sum_{j=k}^{\infty} \mathcal{U}(\xi_j, v_j) \end{aligned} \tag{10}$$

where $\mathcal{Q} \in \mathbb{R}^{2n\times 2n}$ is the state weight, $\mathcal{Q}_{in} \in \mathbb{R}^{n\times n}$ is a part of $\mathcal{Q}$, and $\mathcal{R} \in \mathbb{R}^{m\times m}$ is the control weight. They are all positive diagonal matrices. Using the utility function $\mathcal{U}(\xi_k, v_k)$, the cost function obeys the following DT Hamilton–Jacobi–Bellman equation (HJBE):

$$\mathcal{V}^*(\xi_k) = \min_{v_k} \left\{ \mathcal{U}(\xi_k, v_k) + \mathcal{V}^*(\xi_{k+1}) \right\}. \tag{11}$$

According to [19], [21], and [22], the optimal form of Q-function associated with system state and control input is defined as

$$\begin{aligned} \mathcal{Q}^*(\xi_k, v_k) &= \mathcal{U}(\xi_k, v_k) + \mathcal{V}^*(\xi_{k+1}) \\ &= \mathcal{U}(\xi_k, v_k) + \mathcal{Q}^*\left(\xi_{k+1}, v_{k+1}^*\right) \end{aligned} \tag{12}$$

and the optimal tracking policy can be derived as

$$\begin{aligned} v^*(\xi_k) &= \arg\min_{v_k} \mathcal{Q}^*(\xi_k, v_k) \\ &= -\frac{1}{2}\mathcal{R}^{-1}G(\xi_k)^T \frac{\partial \mathcal{Q}^*\left(\xi_{k+1}, v_{k+1}^*\right)}{\partial \xi_{k+1}}. \end{aligned} \tag{13}$$

Then, the DT-HJBE utilizing the Q-function is given in the subsequent form

$$\mathcal{Q}^*\left(\xi_k, v_k^*\right) = \xi_k^T \mathcal{Q} \xi_k + \left(v_k^*\right)^T \mathcal{R} v_k^* + \mathcal{Q}^*\left(\xi_{k+1}, v_{k+1}^*\right). \tag{14}$$

However, in the traditional iterative Q-learning approach, the state weight $\mathcal{Q}$ and control weight $\mathcal{R}$ in the utility function $\mathcal{U}$ are determined prior to iteration, which means that the optimal control policy obtained by iteration is unchangeable once it is formed. However, in practical applications, the controller often needs to adjust optimal control policies with different state and control weights according to complex changes in the environmental conditions or control objectives. This article proposes a novel PA-QL algorithm that approximates the solution of the OTC problem while equipping the control policy with the capability for flexible and seamless online adjustment.

## III. Derivation and Implementation of the PA-QL Algorithm

In this section, a novel PA-QL algorithm is developed, which endows the control policy with the capability of flexible and seamless online adjustment while maintaining optimality. Then, the implementation process together with hyperparameter selection and complexity analysis is presented.

### A. Derivation of the PA-QL Algorithm

Incorporating the control weight matrix $\mathcal{R}$ to the input of the optimal Q-function (12), the policy-adjustable Q-function can be written as

$$\begin{aligned} \mathcal{Q}^*(\xi_k, \mathcal{R}_d, v_k) &= \xi_k^T \mathcal{Q} \xi_k + v_k^T \mathcal{R} v_k + \mathcal{Q}^*\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^*\right) \\ &= \mathcal{U}(\xi_k, \mathcal{R}_d, v_k) + \mathcal{Q}^*\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^*\right) \end{aligned} \tag{15}$$

where $\mathcal{R} = \text{diag}(\mathcal{R}_d) = \text{diag}([r_{11}, \ldots, r_{mm}]^T)$ and $\mathcal{R}_d$ denotes the vector consisting of the diagonal elements of $\mathcal{R}$.

Then, the optimal control policy is obtained by

$$\begin{aligned} v^*(\xi_k, \mathcal{R}_d) &= \arg\min_{v_k} \mathcal{Q}^*(\xi_k, \mathcal{R}_d, v_k) \\ &= -\frac{1}{2}\mathcal{R}^{-1}G(\xi_k)^T \frac{\partial \mathcal{Q}^*\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^*\right)}{\partial \xi_{k+1}}. \end{aligned} \tag{16}$$

Consequently, the DT-HJBE utilizing the policy-adjustable Q-function is given as

$$Q^*\left(\xi_k, \mathcal{R}_d, v_k^*\right) = \xi_k^T \mathcal{Q} \xi_k + \left(v_k^*\right)^T \mathcal{R} v_k^* + Q^*\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^*\right). \quad (17)$$

*Remark 1:* While classical Q-learning fixes the control weight matrix $\mathcal{Q}$ and $\mathcal{R}$ during training, making post-training adaptation infeasible, the proposed PA-QL algorithm introduces a structural reformulation by explicitly incorporating the diagonal weight vector $\mathcal{R}_d$ as an augmented neural input to both the Q-function and the control policy. This reformulation enables the learned policy $v^*(\xi_k, \mathcal{R}_d)$ to adapt to different environmental conditions and control objectives even after training. Unlike classical frameworks that require retraining when the performance index changes, PA-QL supports online adjustment of control policy in response to varying cost functions, without compromising optimality. This innovation effectively decouples policy learning from a fixed cost function and offers a practical mechanism for adaptive optimal control under uncertain or dynamic environments.

*Remark 2:* It is worth noting that in (17), only the control weight $\mathcal{R}$ is treated as a variable during the iterative process, while the state weight $\mathcal{Q}$ is fixed as a constant. This design is theoretically justified by the homogeneity of the quadratic cost function. Mathematically, minimizing the cost function yields the identical optimal control policy as minimizing a scaled cost function for any positive scalar. This implies that the optimal policy depends fundamentally on the relative ratio between the state weight and the control weight. Consequently, fixing $\mathcal{Q}$ to establish a constant structural preference among state variables while varying only $\mathcal{R}_d$ is sufficient to effectively regulate the tradeoff between regulation performance and control energy. This approach significantly reduces the input dimension of the NNs and the computational complexity of training, without compromising the ability to adjust the global control aggressiveness.

### B. NN Implementation of the PA-QL Algorithm

*1) Model NN:* The inputs of model NN are $x_k$ and $u_k$, and the output is given as follows:

$$\hat{x}_{k+1} = (\omega_{m,2})^T \Gamma\left((\omega_{m,1})^T I_{m,k}\right) \quad (18)$$

where $\omega_{m,1} \in \mathbb{R}^{(n+m)\times H_m}$ and $\omega_{m,2} \in \mathbb{R}^{H_m \times n}$ denote the input-hidden and hidden-output weight of model NN, respectively; $H_m$ is the number of neurons in the hidden layer of model NN; $I_{m,k} = [x_k^T, u_k^T]^T$ is the input vector; and $\Gamma(\cdot)$ is the activation function.

*2) Inverse Dynamic NN:* The inputs of inverse dynamic NN are $x_k, x_{k+1}$ and the output is given as follows:

$$\hat{u}_k = (\omega_{d,2})^T \Gamma\left((\omega_{d,1})^T I_{d,k}\right) \quad (19)$$

where $\omega_{d,1} \in \mathbb{R}^{(2n)\times H_d}$ and $\omega_{d,2} \in \mathbb{R}^{H_d \times m}$ denote the input-hidden and hidden-output weight of inverse dynamic NN, respectively, and $I_{d,k} = [x_k^T, x_{k+1}^T]^T$ is the input vector.

The training processes of model NN and inverse dynamic NN have been given in many papers, such as [21] and [22]. Thus, it will not be discussed in detail in this article.

*3) Critic NN:* The inputs of critic NN are $\xi_k, \mathcal{R}_d, v_k^{i-1}$ and the output is given as follows:

$$\hat{Q}^i\left(\xi_k, \mathcal{R}_d, v_k^{i-1}\right) = \left(\omega_{c,2}^i\right)^T \Gamma\left(\left(\omega_{c,1}^i\right)^T I_{c,k}\right) \quad (20)$$

where $\omega_{c,1}^i \in \mathbb{R}^{(2n+2m)\times H_c}$ and $\omega_{c,2}^i \in \mathbb{R}^{H_c \times 1}$ denote the input-hidden and hidden-output weight of critic NN, respectively; $H_c$ is the number of neurons in the hidden layer of critic NN; superscript $i$ denotes the actor–critic iteration index; and $I_{c,k} = [\xi_k^T, \mathcal{R}_d^T, (v_k^{i-1})^T]^T$ is the input vector.

Theloss function minimized during the training of the critic NN is defined as

$$E_{c,k}^i = \frac{\|e_{c,k}^i\|_2^2}{2} \quad (21)$$

where $e_{c,k}^i = \hat{Q}^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) - Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ and $\|\cdot\|_2$ is the Euclidean norm of the vector.

Set the learning rate of critic NN as $\alpha_c$. The weights $\omega_{c,1}^i$ and $\omega_{c,2}^i$ are updated using the gradient descent method as follows:

$$\omega_{c,2}^{i+1} = \omega_{c,2}^i - \alpha_c \frac{\partial E_{c,k}^i}{\partial \omega_{c,2}^i}, \quad \omega_{c,1}^{i+1} = \omega_{c,1}^i - \alpha_c \frac{\partial E_{c,k}^i}{\partial \omega_{c,1}^i}.$$

Let $\hat{Q}^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) \equiv \hat{Q}_k^i(\cdot)$, and the explicit gradients of the loss function with respect to the weights are derived as follows:

$$\frac{\partial E_{c,k}^i}{\partial \omega_{c,2}^i} = \frac{\partial E_{c,k}^i}{\partial e_{c,k}^i} \frac{\partial e_k^i}{\partial \hat{Q}_k^i(\cdot)} \frac{\partial \hat{Q}_{c,k}^i(\cdot)}{\partial \omega_{c,2}^i}$$
$$\frac{\partial E_{c,k}^i}{\partial \omega_{c,1}^i} = \frac{\partial E_{c,k}^i}{\partial e_{c,k}^i} \frac{\partial e_{c,k}^i}{\partial \hat{Q}_k^i(\cdot)} \frac{\partial \hat{Q}_k^i(\cdot)}{\partial \Gamma(\cdot)} \frac{\partial \Gamma(\cdot)}{\partial \omega_{c,1}^i}.$$

*4) Actor NN:* The inputs of actor NN are $\xi_k, \mathcal{R}_d$ and the output is given as follows:

$$\hat{v}^i(\xi_k, \mathcal{R}_d) = \left(\omega_{a,2}^i\right)^T \Gamma\left((\omega_{a,1}^i)^T I_{a,k}\right) \quad (22)$$

where $\omega_{a,1}^i \in \mathbb{R}^{(2n+m)\times H_a}$ and $\omega_{a,2}^i \in \mathbb{R}^{H_a \times m}$ denote the input-hidden and hidden-output weight of actor NN, respectively; $H_a$ is the number of neurons in the hidden layer of actor NN; and $I_{a,k} = [\xi_k^T, \mathcal{R}_d^T]^T$ is the input vector.

The loss function minimized during the training of the actor NN is defined as

$$E_{a,k}^i = \frac{\|e_{a,k}^i\|_2^2}{2} \quad (23)$$

where $e_{a,k}^i = \hat{v}^i(\xi_k, \mathcal{R}_d) - v^i(\xi_k, \mathcal{R}_d)$.

Set the learning rate of actor NN as $\alpha_a$. The weights $\omega_{a,1}^i$ and $\omega_{a,2}^i$ are updated using the gradient descent method as follows:

$$\omega_{a,2}^{i+1} = \omega_{a,2}^i - \alpha_a \frac{\partial E_{a,k}^i}{\partial \omega_{a,2}^i}, \quad \omega_{a,1}^{i+1} = \omega_{a,1}^i - \alpha_a \frac{\partial E_{a,k}^i}{\partial \omega_{a,1}^i}.$$

The explicit gradients of the loss function with respect to the weights are derived as follows:

$$\frac{\partial E_a^i}{\partial \omega_{a,2}^i} = \frac{\partial E_a^i}{\partial e_{a,k}^i} \frac{\partial e_{a,k}^i}{\partial \hat{v}^i(\xi_k, \mathcal{R}_d)} \frac{\partial \hat{v}^i(\xi_k, \mathcal{R}_d)}{\partial \omega_{a,2}^i}$$
$$\frac{\partial E_a^i}{\partial \omega_{a,1}^i} = \frac{\partial E_a^i}{\partial \hat{e}_{a,k}^i} \frac{\partial e_{a,k}^i}{\partial \hat{v}^i(\xi_k, \mathcal{R}_d)} \frac{\partial \hat{v}^i(\xi_k, \mathcal{R}_d)}{\partial \Gamma(\cdot)} \frac{\partial \Gamma(\cdot)}{\partial \omega_{a,1}^i}.$$

By integrating the above four NNs, the framework of the PA-QL algorithm is shown in Algorithm 1.

*Remark 3:* Since the proposed PA-QL algorithm increases the convergence difficulty of Q-learning, we adopt a variant that incorporates a model NN to estimate the system transition, following the approach in [21] and [22]. Specifically, in (24) and (25), the state vector $\xi_{k+1}$ of augmented system in $Q^i(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^{i-1})$ is approximated via the model NN. Although the inclusion of the model NN incurs extra computational cost during training, it enables the Q-function and control policy to be iteratively updated with respect to the current state $x_k$, rather than relying solely on successor states $x_{k+1}$. This modification improves convergence stability, particularly in offline learning settings with limited or uneven data coverage, and serves as a reliable foundation for implementing the proposed PA-QL algorithm. Moreover, this approach enhances the robustness of the learning process against biased control input distributions and the presence of outliers in the dataset, further supporting its applicability in practical offline scenarios.

### C. Hyperparameter Selection and Complexity Analysis

In practice, the hyperparameter selection follows a balanced strategy to ensure both stability and efficiency. The learning rates are selected to trade off convergence speed and stability, with smaller values ensuring stable updates and larger values accelerating training;

**Algorithm 1** PA Q-Learning Algorithm

**Initialization:**

1: Sample an offline dataset $\mathbb{X} = \{x_k, u_k, x_{k+1}\}^M$, where $M$ represents the number of data, and train the model NN and the inverse dynamic NN.

2: Sample a desired trajectory dataset $\mathbb{D} = \{d_k, d_{k+1}\}^M$ and a control weight dataset $\mathbb{R} = \{\mathcal{R}_d\}^M$ and obtain the training dataset $\mathbb{T} = \{\xi_k, \mathcal{R}_d\}^M$.

3: Give the maximal iteration step $i_{max}$ and the iteration threshold $\chi$. Define $Q_k^i \equiv Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$, Set $Q^0(\cdot) = 0$ and $i = 1$.

4: Train the actor NN by solving the initial control policy:

$$v_k^0 = \arg\min_{v_k} \left(\xi_k^T \mathcal{Q} \xi_k + v_k^T \mathcal{R} v_k + Q^0(\cdot)\right).$$

5: Train the critic NN by solving the initial Q-function:

$$Q_k^1(\xi_k, \mathcal{R}_d, v_k^0) = \xi_k^T \mathcal{Q} \xi_k + (v_k^0)^T \mathcal{R} v_k^0 + Q^0(\cdot)$$

**Iteration:**

6: Train the actor NN by updating the iterative control policy:

$$v_k^i = \arg\min_{v_k} \left(\xi_k^T \mathcal{Q} \xi_k + (v_k)^T \mathcal{R} v_k + Q_{k+1}^i(\cdot)\right). \quad (24)$$

7: Train the critic NN by update the iterative Q-function:

$$Q_k^{i+1}(\cdot) = \xi_k^T \mathcal{Q} \xi_k + \left(v_k^i\right)^T \mathcal{R} v_k^i + Q_{k+1}^i(\cdot) \quad (25)$$

8: If $|Q_k^{i+1}(\cdot) - Q_k^i(\cdot)| \le \chi$ or $i \ge i_{max}$, then go to step 9; else let $i = i + 1$ and return to step 6.

9: If $Q_k^{i+1}(\cdot) - Q_k^i(\cdot) < \mathcal{U}(\xi_k, \mathcal{R}_d, v_k^i)$, then save control policy $v^{i-1}$; else let $i = i + 1$ and return to step 6.

typical values lie within $[10^{-4}, 10^{-2}]$. The iteration threshold $\chi$ controls the stopping criterion of the Q-function update, where a smaller $\chi$ yields higher solution accuracy at the cost of longer training time. The maximal iteration step $i_{\max}$ serves as a safeguard against infinite loops and is generally set sufficiently large so that convergence occurs well before this limit. Moreover, to mitigate overfitting due to increased input dimensionality, a compact network architecture is selected to ensure a high sample-to-parameter ratio, complemented by randomized input sampling across the continuous weight space. This configuration promotes the learning of underlying functional mappings rather than the memorization of specific data points, thereby enhancing generalization. Regarding the dataset size $M$, the incorporation of the model NN significantly improves data efficiency, enabling stable convergence with a relatively compact dataset and reducing the dependency on massive data collection.

Define the epochs for the network as $E_m, E_d, E_c$, and $E_a$; then, the overall computational complexity is

$$\mathcal{O}\left(M \cdot (E_m P_m + E_d P_d + i_{\max}(E_c P_c + E_a P_a))\right)$$

which covers the pretraining of model NN, inverse dynamic NN, and the iterative actor–critic updates. Here, the number of parameters for each network is calculated as

$$\begin{cases} P_m = (n+m)H_m + H_m + H_m n + n \\ P_d = 2nH_d + H_d + H_d m + m \\ P_c = (2n+2m)H_c + H_c + H_c + 1 \\ P_a = (2n+m)H_a + H_a + H_a m + m. \end{cases}$$

The total memory complexity is represented as

$$\mathcal{O}\left(P_m + P_d + P_c + P_a\right) + \mathcal{O}\left(B \cdot (2n+2m)\right) + \mathcal{O}\left(M \cdot (4n+2m)\right)$$

accounting for network parameter storage, mini-batch activations during backpropagation, and dataset caching. Here, $n$ and $m$ denote the state and input dimensions, $M$ is the dataset size, and $B$ is the mini-batch size.

## IV. Properties of the PA-QL Algorithm

In this section, the convergence and stability of the PA-QL algorithm are developed. Before presenting the convergence analysis, the following two lemmas are introduced.

*Lemma 1:* Let $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i)$ be derived from (25), $Q^0(\cdot) = 0$ and the function $\Psi^{i+1}(\xi_k, \mathcal{R}_d, \mu_k^i)$ is generated by an arbitrary control policy $\mu_k^i$ as follows:

$$\Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^i\right) = \mathcal{U}\left(\xi_k, \mathcal{R}_d, \mu_k^i\right) + \Psi^i\left(\xi_{k+1}, \mathcal{R}_d, \mu_{k+1}^{i-1}\right). \quad (26)$$

Then, the following three conclusions hold.

1) When $\Psi^0(\cdot) = 0$ is satisfied, then $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \le \Psi^{i+1}(\xi_k, \mathcal{R}_d, \mu_k^i)$ holds.
2) If $\mu_k^i = \mu_k^\pi$ is an admissible control policy, there exists an upper bound $\Lambda(\xi_k)$ such that $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \le \Psi^{i+1}(\xi_k, \mathcal{R}_d, \mu_k^i) \le \Lambda(\xi_k)$.
3) If $\mu_k^i = v_k^*$ is the optimal control policy and (17) is solvable, then $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \le Q^*(\xi_k, \mathcal{R}_d, v_k^*) \le \Lambda(\xi_k)$.

*Proof:* According to Algorithm 1, $Q_k^{i+1}(\cdot)$ is obtained by minimizing the right side of (25) related to the control policy $v_k^i$ defined in (24); however, $\Psi_k^{i+1}(\cdot)$ is obtained by an arbitrary control policy$\mu_k^i$ . Then, it is obvious that $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \le \Psi^{i+1}(\xi_k, \mathcal{R}_d, \mu_k^i)$ is satisfied.

Assume that $\mu_k^\pi$ is an admissible control policy, and then (26) can be rewritten as

$$\Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) = \mathcal{U}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) + \Psi^i\left(\xi_{k+1}, \mathcal{R}_d, \mu_{k+1}^\pi\right). \quad (27)$$

Then, it is easy to derive that

$$\begin{aligned} &\Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) - \Psi^i\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \\ &= \Psi^i\left(\xi_{k+1}, \mathcal{R}_d, \mu_{k+1}^\pi\right) - \Psi^{i-1}\left(\xi_{k+1}, \mathcal{R}_d, \mu_{k+1}^\pi\right) \\ &\quad \vdots \\ &= \Psi^1\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right) - \Psi^0\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right) \\ &= \Psi^1\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right). \end{aligned} \quad (28)$$

According to (28), one has

$$\begin{aligned} &\Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \\ &= \Psi^1\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right) + \Psi^i\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \\ &= \Psi^1\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right) + \Psi^1\left(\xi_{k+i-1}, \mathcal{R}_d, \mu_{k+i-1}^\pi\right) \\ &\quad + \Psi^{i-1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \\ &\quad \vdots \\ &= \Psi^1\left(\xi_{k+i}, \mathcal{R}_d, \mu_{k+i}^\pi\right) + \Psi^1\left(\xi_{k+i-1}, \mathcal{R}_d, \mu_{k+i-1}^\pi\right) \\ &\quad + \cdots + \Psi^1\left(\xi_{k+1}, \mathcal{R}_d, \mu_{k+1}^\pi\right) + \Psi^1\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \\ &= \sum_{j=0}^{i} \left\{\mathcal{U}\left(\xi_{k+j}, \mathcal{R}_d, \mu_{k+j}^\pi\right)\right\}. \end{aligned} \quad (29)$$

Since $\mu_k^\pi$ is an admissible control policy, when $j \to \infty$, it is clear that $\mathcal{U}(\xi_{k+j}, \mathcal{R}_d, \mu_{k+j}^\pi) \to 0$, then

$$\Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \le \sum_{j=0}^{\infty} \left\{\mathcal{U}\left(\xi_{k+j}, \mathcal{R}_d, \mu_{k+j}^\pi\right)\right\}. \quad (30)$$

Let $\sum_{j=0}^{\infty} \left\{\mathcal{U}(\xi_{k+j}, \mathcal{R}_d, \mu_{k+j}^\pi)\right\} = \Lambda(\xi_k)$, and one has

$$Q^{i+1}\left(\xi_k, \mathcal{R}_d, v_k^i\right) \le \Psi^{i+1}\left(\xi_k, \mathcal{R}_d, \mu_k^\pi\right) \le \Lambda(\xi_k). \quad (31)$$

The optimal control policy $v_k^*$ is a special case of the admissible control policy$\mu_k^\pi$ , (31) can be rewritten as

$$Q^{i+1}\left(\xi_k, \mathcal{R}_d, v_k^i\right) \le Q^*\left(\xi_k, \mathcal{R}_d, v_k^*\right) \le \Lambda(\xi_k). \quad (32)$$

The proof is completed. ■

*Lemma 2:* Suppose $Q^i(\xi_k, \mathcal{R}_d, v_k^\pi)$ and $v_k^i$ to be defined by (25) and (24), respectively. If $Q^0(\cdot) = 0$, then $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \geq Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ holds.

*Proof:* The proof is derived by mathematical induction.

1) For $i = 1$, since $Q^0(\cdot) = 0$ and $\mathcal{Q}$ and $\mathcal{R}$ are positive definite matrices, we have

$$Q^1(\xi_k, \mathcal{R}_d, v_k^0) - Q^0(\cdot) = \xi_k^T \mathcal{Q} \xi_k + (v_k^0)^T \mathcal{R} v_k^0 \geq 0. \tag{33}$$

Thus, $Q^1(\xi_k, \mathcal{R}_d, v_k^0) \geq Q^0(\cdot)$ holds.

2) Assume that $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) \geq Q^{i-1}(\xi_k, \mathcal{R}_d, v_k^{i-2})$ holds for $i \geq 1$. According to (25), we have

$$\begin{aligned} &Q^{i+1}\left(\xi_k, \mathcal{R}_d, v_k^i\right) \\ &= \min_{v_k} \left\{ \mathcal{U}\left(\xi_k, \mathcal{R}_d, v_k\right) + Q^i\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^{i-1}\right) \right\} \\ &\geq \min_{v_k} \left\{ \mathcal{U}\left(\xi_k, \mathcal{R}_d, v_k\right) + Q^{i-1}\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^{i-2}\right) \right\} \\ &= Q^i\left(\xi_k, \mathcal{R}_d, v_k^{i-1}\right). \end{aligned} \tag{34}$$

Therefore, $Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \geq Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ holds. The proof is completed. ■

*Theorem 1:* If $Q^0(\cdot) = 0$ is satisfied, when $i \to \infty$, the iterative Q-function $Q_k^i(\cdot)$ obtained by (25) converges to the optimal Q-function $Q_k^*(\cdot)$ and the iterative control policy $v_k^i$ derived from (24) converges to the optimal control policy $v_k^*$.

*Proof:* From Lemmas 1 and 2, we have $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) \leq Q^{i+1}(\xi_k, \mathcal{R}_d, v_k^i) \leq \Lambda(\xi_k)$. Since the sequence $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ is nondecreasing and bounded, according to the monotone convergence theorem, $\lim_{i\to\infty} Q_k^i(\cdot)$ exists. Taking the limit as $i \to \infty$ on both sides of (24) and (25), by the uniqueness of the solution to the Bellman optimality equation, we have $\lim_{i\to\infty} Q_k^i = Q_k^*$. Consequently, we obtain $\lim_{i\to\infty} v_k^i = v_k^*$. ■

At the point of convergence analysis, it can be shown that under the optimal control policy $v^*$, the system state $\xi_k$ is asymptotically stable, and its Q-function satisfies $Q^*(\xi_k, \mathcal{R}_d, v_k^*) = \sum_{i=k}^{\infty} \mathcal{U}(\xi_i, \mathcal{R}_d, v_i^*)$. However, the condition $v^i = v^*$ requires $i \to \infty$, which is unattainable in practical applications since the algorithm must terminate within a finite number of iterations. Therefore, it is essential to analyze the stability of $v^i$ within a finite number of iterations for the PA-QL algorithm.

*Theorem 2:* Consider the Q-function $Q^{i+1}$ and the control policy $v^i$ defined by (24) and (25), respectively. If the following inequality:

$$Q^{i+1}\left(\xi_k, \mathcal{R}_d, v_k^i\right) - Q^i\left(\xi_k, \mathcal{R}_d, v_k^{i-1}\right) < \mathcal{U}\left(\xi_k, \mathcal{R}_d, v_k^i\right) \tag{35}$$

holds, the system state $\xi_k$ is asymptotically stable under the control policy $v^{i-1}$.

*Proof:* To analyze the stability of the system under the finite-iteration control policy $v^{i-1}$, the following Lyapunov function is defined:

$$V(e_k) = e_k^T \mathcal{Q}_{in} e_k + \left(v_k^{i-1}\right)^T \mathcal{R} v_k^{i-1} + Q^{i-1}\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^{i-2}\right). \tag{36}$$

It can be readily deduced from (25) and (36) that $V(e_k) = Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ holds. Given that $Q^0(\cdot) = 0$ and $Q^1(\xi_k, \mathcal{R}_d, v_k^0) = \xi_k^T \mathcal{Q} \xi_k + (v_k^0)^T \mathcal{R} v_k^0$, we can deduce that $Q^1(\xi_k, \mathcal{R}_d, v_k^0) > 0$. According to Lemma 2, one has $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) > 0$ that holds for any $i \geq 1$. Thus, we have $V(e_k) > 0$ such that $\xi_k \neq 0$. Moreover, when $e_k = 0$, it is easy to know that $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1}) = 0$ hold. Hence, we have $V(e_k) = 0$ such that $e_k = 0$.

Substituting (25) into (35), we obtain

$$Q^i\left(\xi_{k+1}, \mathcal{R}_d, v_{k+1}^{i-1}\right) - Q^i\left(\xi_k, \mathcal{R}_d, v_k^{i-1}\right) < 0. \tag{37}$$

From this, we conclude that $V(e_{k+1}) - V(e_k) < 0$. Furthermore, by Lemma 1, $Q^i(\xi_k, \mathcal{R}_d, v_k^{i-1})$ has an upper bound $\Lambda(\xi_k)$. Then, we can obtain $\lim_{k\to\infty} V(e_k) = 0$, which proves that, under policy $v^{i-1}$, $\xi_k$ in system is asymptotically stable. ■

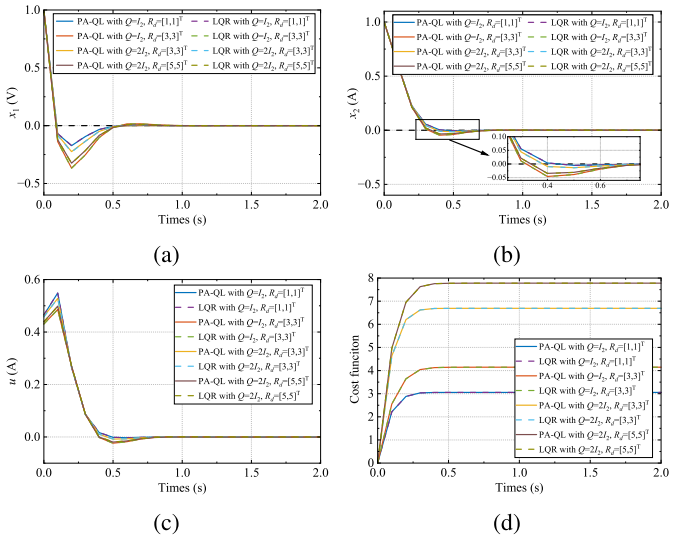



Fig. 1. Example 1: simulation results. (a) State trajectory $x_1$, (b) state trajectory $x_2$, (c) control input $u$, and (d) convergence trajectories of cost function.

## V. Simulation Studies

### A. Example 1

Since the exact analytical solution for the optimal control of general nonlinear systems is typically unavailable, a linear time-invariant (LTI) system is adopted in Example 1 to validate the optimality capabilities of the proposed algorithm. The selected system is an RLC circuit, which can be described as follows:

$$\begin{bmatrix} x_{1,k+1} \\ x_{2,k+1} \end{bmatrix} = \begin{bmatrix} 1 & -\dfrac{\Delta t}{C} \\ \dfrac{\Delta t}{L} & 1 - \dfrac{R \cdot \Delta t}{L} \end{bmatrix} \begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} + \begin{bmatrix} x_{1,k} \\ x_{2,k} \end{bmatrix} u_k \tag{38}$$

where $x_{1,k}$ and $x_{2,k}$ represent the voltage of the capacitor and the current of the inductor, respectively; $u_k$ denotes the control input of the current source; $R$, $C$, and $L$ are the resistance, capacitance, and inductance, respectively; and $\Delta t$ denotes the sampling interval. These parameters are selected as $\Delta t = 0.1\,\text{s}$, $C = 0.05\,\text{F}$, $L = 0.5\,\text{H}$, and $R = 3\,\Omega$.

The model NN adopts a 3-10-2 architecture, while the inverse dynamics NN is configured as 4-12-1. With $\mathcal{Q}_{in} = I_2$, where $I_2$ denotes the $2 \times 2$ identity matrix, the critic NN has a 6-24-1 architecture, while the actor NN is structured as 4-16-1. All NNs use a learning rate of 0.005 and the threshold $\chi$ is set to $10^{-5}$. To create the training dataset $\mathbb{T}$, we collect 1000 data pairs from the system. Afterward, the PA-QL Algorithm 1 is applied to derive the optimal control policy. The desired trajectory is defined as $d_k = [0, 0]^T$. It is worth noting that the control task in this example corresponds to a regulation problem, where the steady control input is zero, i.e.,$u_s = 0$. As a result, Assumption 1 does not need to be satisfied in this specific case.

For such linear systems with quadratic cost functions, the optimal control policy can be explicitly obtained via the LQR by solving the algebraic Riccati equation (ARE). This provides a reliable reference for evaluating the performance of the proposed method. After obtaining the controller through the proposed training process, two experiments are conducted to validate its optimality under different input weight matrices $\mathcal{R}_d$ and initial states $x_0$.

In the first set of experiments, the initial state is fixed at $x_0 = [1, 1]^T$. To rigorously verify the optimality of the proposed method against the theoretical benchmark, we evaluated the control performance under distinct cost function configurations. First, to validate the algorithm's consistency under different state penalties, we trained an additional controller with $\mathcal{Q}_{in} = 2I_2$. Second, we tested the proposed algorithm under various assigned input weight

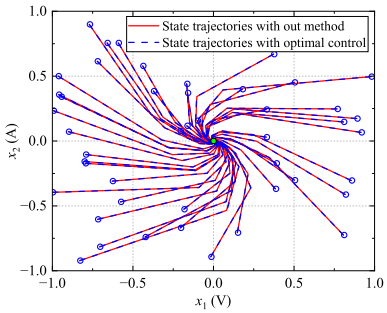
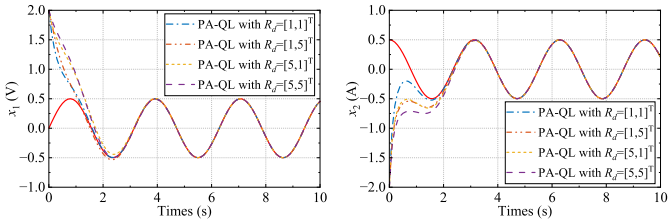



Fig. 2. Example 1: state trajectories of $\mathbb{X}_0$.




(a) (b)

Fig. 3. Example 2: simulation results. (a) State trajectory $x_1$ with different $\mathcal{R}_d$'s and (b) state trajectory $x_2$ with different $\mathcal{R}_d$'s (the red trajectories represent the desired trajectories).

matrices $\mathcal{R}_d$. As shown in Fig. 1, the control input generated by the proposed algorithm coincides exactly with the theoretical optimal control obtained from solving the ARE. This result demonstrates that, even when the input weight matrix $\mathcal{R}_d$ is modified after training, the control policy produced by the proposed algorithm remains optimal. Therefore, the proposed method is capable of achieving optimality under adjustments of the weight matrix $\mathcal{R}_d$ without retraining.

In the second set, a set of initial states $\mathbb{X}_0 = \{x_0 | -1 < x_0^{[1]} < 1, -1 < x_0^{[2]} < 1\}^{50}$ is randomly selected, the input weight matrix is set to $\mathcal{R}_d = [1, 1]^T$. The 50 state trajectories are shown in Fig. 2; for all randomly generated initial states, the control input generated by the proposed algorithm matches exactly with the theoretical optimal control. This further confirms the optimality of the proposed method across a wide range of initial conditions.

## B. Example 2

The Van der Pol oscillator system is considered to demonstrate the effectiveness of the algorithm in a nonlinear system; the system is discretized with a sampling interval of 0.1 s and is described by the following equation:

$$\begin{bmatrix} x_{1,k+1} \\ x_{2,k+1} \end{bmatrix} = \begin{bmatrix} x_{1,k} + 0.1x_{2,k} \\ 1.1x_{2,k} - 0.1\left(x_{1,k} + (x_{1,k})^2 x_{2,k}\right) \end{bmatrix} + \begin{bmatrix} 0.2 & 0 \\ 0 & 0.4 \end{bmatrix} \begin{bmatrix} u_{1,k} \\ u_{2,k} \end{bmatrix}. \tag{39}$$

In addition, the desired trajectory is defined as

$$d_k = 0.5 \begin{bmatrix} \sin(0.2k) \\ \cos(0.2k) \end{bmatrix}. \tag{40}$$

The model NN and the inverse dynamics NN are both configured with a 4-16-2 architecture. With $\mathcal{Q}_{in}$ set as a $2 \times 2$ identity matrix, the critic NN has an 8-32-1 architecture, while the actor NN is structured as 6-24-1. All NNs use a learning rate of 0.002 and the threshold $\chi$ is set to $10^{-5}$. To create the training dataset $\mathbb{T}$, we collect 2000 data pairs from the system. Afterward, the PA-QL Algorithm 1 is applied to derive the optimal control policy.

After the control policy is learned through iterative training, by varying the input $\mathcal{R}_d$, one can observe corresponding changes in the state trajectories shown in Fig. 3. When $\mathcal{R}_d$ takes larger values, the control strategy becomes more conservative and the convergence speed slows down, whereas smaller values of $\mathcal{R}_d$ lead to the opposite effect. This demonstrates that the proposed algorithm can effectively adjust the control policy by modifying the input $\mathcal{R}_d$ even after the offline iterations, indicating its capability to retain online adaptability of the control strategy.

To better demonstrate the capability of the proposed algorithm to adjust control policies online, an error-driven adjustment mechanism for the control weight matrix is introduced. This strategy is commonly used in online optimal control schemes such as LQR and MPC [32]. The core idea is to reduce the control weight when the tracking error is large, thereby allowing more aggressive control inputs to quickly drive the system toward the target signal. Conversely, when the tracking error is small, the control weight is increased to suppress unnecessary control effort and enhance smoothness. Defining $\tau$ as the adaptive factor for $\mathcal{R}_d$, the control weight matrix in the cost function is rewritten as $\tau \cdot \mathcal{R}_d$. The adaptive rate of the control weight matrix with respect to the tracking error is designed as follows:

$$\tau = 1 + 4 \cdot e^{-\|x_k - d_k\|_2}. \tag{41}$$

For comparison, a baseline approach based on the conventional Q-learning algorithm is constructed [22]. In this method, several separate policies are trained offline using a fixed control weight matrix $\mathcal{R}_d$, each corresponding to a specific performance index. During execution, an error-driven switching strategy is adopted, where the control policy is selected from among these pretrained policies according to the current tracking error. This conventional method serves as a reference for evaluating the effectiveness and adaptability of the proposed PA-QL approach under dynamic performance requirements. The switching conditions for selecting among the pretrained control policies are defined as follows:

$$\tau = \begin{cases} 1, & \|x_k - d_k\|_2 > 1.5 \\ 2, & 1 < \|x_k - d_k\|_2 \le 1.5 \\ 3, & 0.5 < \|x_k - d_k\|_2 \le 1 \\ 4, & 0.1 < \|x_k - d_k\|_2 \le 0.5 \\ 5, & \|x_k - d_k\|_2 \le 0.1. \end{cases} \tag{42}$$

Fig. 4(a), (b), (e), and (f) depicts the state trajectories and control input under the different control strategies. It can be observed that both the policy-switching method and the proposed error-driven parameter adaptation approach based on PA-QL exhibit similar behaviors. When the tracking error is large, both methods generate more aggressive control inputs to quickly drive the system toward the desired trajectory. As the error diminishes, the control effort is gradually reduced, leading to smoother convergence. Ultimately, both methods are able to achieve accurate trajectory tracking, as shown in Fig. 4(d).

However, as illustrated in Fig. 4(e) and (f), the policy-switching-based method exhibits sharp transitions in the control signal at the switching points in Fig. 4(c). These abrupt jumps are inherent to the stepwise nature of policy switching and may lead to undesirable effects on both the controller and the system. In contrast, the proposed PA-QL approach achieves error-driven control in a smooth and continuous manner. By adjusting the control weight matrix adaptively based on the real-time error, the control input remains stable without introducing abrupt changes. This makes PA-QL more suitable for real-time applications that require both responsiveness and control smoothness.

To further evaluate performance against online optimization methods, we compared PA-QL with an adaptive MPC (prediction horizon $N = 10$) using identical error-driven objective functions. As shown in Fig. 5, PA-QL achieves tracking precision and stability remarkably similar to the MPC, with similar trajectories. However, PA-QL demonstrates a decisive advantage in online computational efficiency. Simulations were conducted on an Intel Core i7-14700KF CPU with 32-GB RAM; the average computation time per step for PA-QL is only $8.024 \times 10^{-6}$ s, compared to $3.60 \times 10^{-3}$ s for MPC, representing

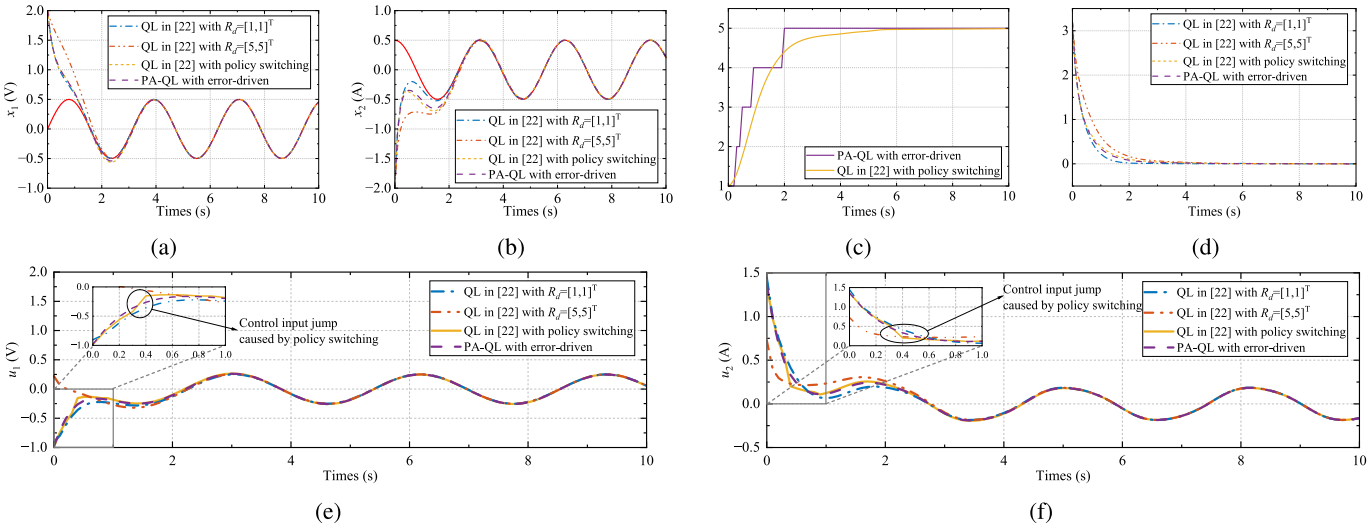



Fig. 4. Example 2: simulation results. (a) State trajectory $x_1$, (b) state trajectory $x_2$, (c) input weight matrix $R_d$, (d) state error $\|x_k - d_k\|_2$, (e) control input $u_1$, and (f) control input $u_2$ (the red trajectories represent the desired trajectories).

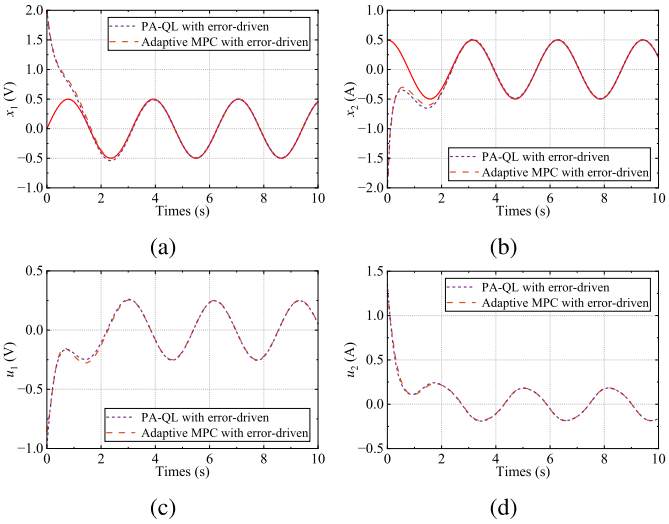



Fig. 5. Example 2: simulation results. (a) State trajectory $x_1$, (b) state trajectory $x_2$, (c) control input $u_1$, and (d) control input $u_2$ (the red trajectories represent the desired trajectories).

a reduction of nearly three orders of magnitude. This comparison highlights that PA-QL effectively bridges the gap between traditional ADP and MPC; it retains the rapid execution speed of offline-trained policies while offering the online parameter adjustment capability typically exclusive to online optimization-based methods.

It is worth noting that various strategies exist for adaptive adjustment of control weights. In this work, an error-driven adaptation mechanism is employed primarily to demonstrate the capability of the proposed PA-QL algorithm to support online weight adjustment within the ADP framework. With the proposal of the PA-QL algorithm, a broad class of adaptive weighting strategies that were previously applied in optimal control methods such as LQR and MPC can now be incorporated into the ADP framework. This advancement establishes a conceptual and algorithmic connection between traditional optimal control and ADP, thereby enhancing the applicability and flexibility of ADP in real-world control tasks.

## VI. Conclusion

This article introduced a novel PA-QL algorithm, whose key innovation lies in embedding control weights directly into the Q-function so that the learned policy can be flexibly and seamlessly adjusted online even after offline training. Simulation results verified its optimality and adaptability across different scenarios. These results highlight the potential of PA-QL to bridge traditional optimal control and ADP, providing a framework for adaptive control in dynamic and uncertain environments. Future work will focus on establishing a rigorous theoretical analysis under NN approximation error propagation to solidify the foundation, alongside extending the framework to broader practical systems with real-time adaptive capabilities.

## References

[1] L. P. Kaelbling, M. L. Littman, and A. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, May 1996.

[2] D. Bertsekas, *Reinforcement Learning and Optimal Control*, vol. 1. Belmont, MA, USA: Athena Scientific, 2019.

[3] Y. Jiang, L. Liu, and G. Feng, "Adaptive optimal control of networked nonlinear systems with stochastic sensor and actuator dropouts based on reinforcement learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3107–3120, Mar. 2024.

[4] H. Shen, C. Peng, H. Yan, and S. Xu, "Data-driven near optimization for fast sampling singularly perturbed systems," *IEEE Trans. Autom. Control*, vol. 69, no. 7, pp. 4689–4694, Jul. 2024.

[5] P. J. Werbos, "Approximate dynamic programming for real-time control and neural modeling," in *Handbook of Intelligent Control*. Van Nostrand, 1992, pp. 493–526. [Online]. Available: https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=Approximate+dynamic+programming+for+realtime+control+and+neural+modeling&btnG=

[6] W. B. Powell, *Approximate Dynamic Programming: Solving the Curses of Dimensionality*, vol. 703. Hoboken, NJ, USA: Wiley, 2007.

[7] A. Al-Tamimi, F. L. Lewis, and M. Abu-Khalaf, "Discrete-time nonlinear HJB solution using approximate dynamic programming: Convergence proof," *IEEE Trans. Syst., Man, Cybern., B (Cybern.)*, vol. 38, no. 4, pp. 943–949, Aug. 2008.

[8] D. Wang, N. Gao, D. Liu, J. Li, and F. L. Lewis, "Recent progress in reinforcement learning and adaptive dynamic programming for advanced control applications," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 1, pp. 18–36, Nov. 2023.

[9] M. Lin, B. Zhao, and D. Liu, "Optimal learning output tracking control: A model-free policy optimization method with convergence analysis," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 3, pp. 5574–5585, Mar. 2025.

[10] F. L. Lewis and D. Liu, *Reinforcement Learning and Approximate Dynamic Programming for Feedback Control*. Hoboken, NJ, USA: Wiley, 2013.

[11] S. Song, M. Zhao, D. Gong, and M. Zhu, "Convergence and stability analysis of value iteration Q-learning under non-discounted cost for discrete-time optimal control," *Neurocomputing*, vol. 606, Nov. 2024, Art. no. 128370.

[12] J. Wang, J. Wu, H. Shen, J. Cao, and L. Rutkowski, "Fuzzy $H_\infty$ control of discrete-time nonlinear Markov jump systems via a novel hybrid reinforcement Q-learning method," *IEEE Trans. Cybern.*, vol. 53, no. 11, pp. 7380–7391, Nov. 2023.

[13] Q. Wei and T. Li, "Constrained-cost adaptive dynamic programming for optimal control of discrete-time nonlinear systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3251–3264, Mar. 2024.

[14] B. Kiumarsi, F. L. Lewis, H. Modares, A. Karimpour, and M.-B. Naghibi-Sistani, "Reinforcement Q-learning for optimal tracking control of linear discrete-time systems with unknown dynamics," *Automatica*, vol. 50, no. 4, pp. 1167–1175, 2014.

[15] J. Y. Lee, J. B. Park, and Y. H. Choi, "Integral Q-learning and explorized policy iteration for adaptive optimal control of continuous-time linear systems," *Automatica*, vol. 48, no. 11, pp. 2850–2859, 2012.

[16] M. Palanisamy, H. Modares, F. L. Lewis, and M. Aurangzeb, "Continuous-time Q-learning for infinite-horizon discounted cost linear quadratic regulator problems," *IEEE Trans. Cybern.*, vol. 45, no. 2, pp. 165–176, Feb. 2015.

[17] Y. Jiang, T. Yang, W. Gao, J. Wu, T. Chai, and F. L. Lewis, "Off-policy reinforcement learning for $H_\infty$ control of linear discrete-time systems with network-induced dropouts," *IEEE Trans. Autom. Control*, vol. 70, no. 12, pp. 8000–8015, Dec. 2025.

[18] C. Li, J. Ding, F. L. Lewis, and T. Chai, "Model-free Q-learning for the tracking problem of linear discrete-time systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 3, pp. 3191–3201, Mar. 2024.

[19] B. Luo, D. Liu, T. Huang, and D. Wang, "Model-free optimal tracking control via critic-only Q-learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 27, no. 10, pp. 2134–2144, Oct. 2016.

[20] Q. Wei, F. L. Lewis, Q. Sun, P. Yan, and R. Song, "Discrete-time deterministic $Q$-learning: A novel convergence analysis," *IEEE Trans. Cybern.*, vol. 47, no. 5, pp. 1224–1237, May 2017.

[21] J. Li, T. Chai, F. L. Lewis, Z. Ding, and Y. Jiang, "Off-policy interleaved $Q$-learning: Optimal control for affine nonlinear discrete-time systems," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 30, no. 5, pp. 1308–1320, May 2019.

[22] S. Song, M. Zhu, X. Dai, and D. Gong, "Model-free optimal tracking control of nonlinear input-affine discrete-time systems via an iterative deterministic Q-learning algorithm," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 1, pp. 999–1012, Jan. 2024.

[23] X. Li, D. Wang, M. Zhao, and J. Qiao, "Reinforcement learning control with n-step information for wastewater treatment systems," *Eng. Appl. Artif. Intell.*, vol. 133, Jul. 2024, Art. no. 108033.

[24] M. Zhao, D. Wang, J. Ren, and J. Qiao, "Integrated online Q-learning design for wastewater treatment processes," *IEEE Trans. Ind. Informat.*, vol. 21, no. 2, pp. 1833–1842, Feb. 2025.

[25] R. Muduli, D. Jena, and T. Moger, "A survey on load frequency control using reinforcement learning-based data-driven controller," *Appl. Soft Comput.*, vol. 166, Nov. 2024, Art. no. 112203.

[26] H. Liu et al., "Load frequency control strategy of island microgrid with flexible resources based on DQN," in *Proc. IEEE Sustain. Power Energy Conf. (iSPEC)*, Dec. 2021, pp. 632–637.

[27] L. N. Tan, N. Gupta, and M. Derawi, "Adaptive dynamic programming and zero-sum game-based distributed control for energy management systems with Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 22371–22385, Dec. 2023.

[28] J. Ye, H. Dong, Y. Bian, H. Qin, and X. Zhao, "ADP-based optimal control for discrete-time systems with safe constraints and disturbances," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 115–128, 2025.

[29] K. Wang, C. Mu, Z. Ni, and D. Liu, "Safe reinforcement learning and adaptive optimal control with applications to obstacle avoidance problem," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 4599–4612, Jul. 2024.

[30] Z. Xu, K. Wang, C. Mu, and T. Qiu, "Safety-critical path planning for obstacle avoidance based on reinforcement learning and control barrier functions," *IEEE Internet Things J.*, vol. 12, no. 23, pp. 51410–51421, Dec. 2025.

[31] O. Saleem, M. Rizwan, and J. Iqbal, "Adaptive optimal control of under-actuated robotic systems using a self-regulating nonlinear weight-adjustment scheme: Formulation and experimental verification," *PLoS ONE*, vol. 18, no. 12, Dec. 2023, Art. no. e0295153.

[32] S. Trimpe, A. Millane, S. Doessegger, and R. D'Andrea, "A self-tuning LQR approach demonstrated on an inverted pendulum," *IFAC Proc. Volumes*, vol. 47, no. 3, pp. 11281–11287, 2014.

[33] K. Ahmed, A. A. Aly, and M. O. Elhabib, "Design of adaptive LQR control based on improved grey wolf optimization for prosthetic hand," *Biomimetics*, vol. 10, no. 7, p. 423, Jun. 2025.

[34] K. S. Narendra, J. Balakrishnan, and M. K. Ciliz, "Adaptation and learning using multiple models, switching, and tuning," *IEEE Control Syst. Mag.*, vol. 15, no. 3, pp. 37–51, Mar. 1995.

[35] W. Lu, P. Zhu, and S. Ferrari, "A hybrid-adaptive dynamic programming approach for the model-free control of nonlinear switched systems," *IEEE Trans. Autom. Control*, vol. 61, no. 10, pp. 3203–3208, Oct. 2016.